ML MODEL DEPLOYMENT STRATEGIES AND MODEL SERVING USING FASTAPI

Dataset: Titanic – Machine Learning from Disaster (Kaggle)

Dataset source: https://www.kaggle.com/competitions/titanic/data

Objective

The objective of this assignment is to train and evaluate a classification model using the labelled Titanic dataset, save the complete preprocessing and prediction pipeline, demonstrate online, batch and streaming inference, and serve the saved model through a REST API built with FastAPI.

1. Dataset and Preprocessing

The assignment uses the train.csv file from the Kaggle Titanic competition. The target column is Survived. PassengerId, Name, Ticket and Cabin are excluded from the model inputs. The selected input features are Pclass, Sex, Age, SibSp, Parch, Fare and Embarked.

Missing numeric values are imputed using the median. Missing categorical values are imputed using the most frequent category. Numeric features are scaled using StandardScaler, and categorical features are converted into numerical features using OneHotEncoder with unknown categories ignored. These preprocessing steps are combined with a Random Forest classifier in a single scikit-learn Pipeline.

2. Model Training and Evaluation

The labelled dataset is split into training and testing portions using an 80:20 split, a fixed random seed of 42, and stratification on the Survived target. The Random Forest classifier uses 200 trees and a fixed random seed of 42. The pipeline is fitted only on the training portion and evaluated on the held-out testing portion using accuracy and a classification report.

The trained pipeline is saved as model.pkl using Joblib. The actual test accuracy should be taken from the output of train_model.py; no estimated or mock accuracy is reported here.

3. Online Inference

Online inference loads model.pkl and selects one record from the reserved held-out records in unseen_passengers.csv. The saved pipeline predicts the passenger's survival class and provides an estimated survival probability. The prediction is generated by the fitted model and is not a statement of historical ground truth.

4. Batch Inference

Batch inference loads the same saved pipeline and predicts 20 reserved passenger records in one operation. The predicted class and its text label are appended to the records and saved to batch_predictions.csv. The output file is used to verify that the batch contains 20 predictions.

5. Streaming Inference

Streaming inference processes 10 reserved passenger records individually. The script sends one record to the model at a time and waits one second between consecutive records to simulate a stream of incoming data. Each record's predicted survival class and estimated survival probability are displayed in the terminal.

6. Model Serving Using FastAPI

The FastAPI application loads the existing model.pkl when the application starts. The GET /health endpoint reports the API health status and whether the model was loaded. The POST /predict endpoint accepts passenger features, validates them using a Pydantic model, and returns the predicted survival class, prediction label and estimated survival probability.

Swagger UI at /docs is used to test a valid prediction request and an invalid request. The invalid request should be rejected with HTTP 422 when its values violate the Pydantic validation rules. A separate Python client, client.py, sends requests to the health and prediction endpoints using the requests library.

7. Observations

1. How does Online Inference differ from Batch Inference?

Online inference processes a single record per request and returns its prediction. Batch inference processes a group of records together and returns predictions for the group. Online inference is useful when an individual result is needed immediately, while batch inference is useful for processing many records in one operation.

2. What is the difference between Streaming Inference and Batch Inference?

Streaming inference processes records one at a time as they arrive, allowing results to be produced continuously. Batch inference collects a set of records and processes them together. In this assignment, streaming is simulated by processing 10 records individually with a one-second delay between consecutive records.

3. Why is FastAPI used for model serving?

FastAPI exposes the trained model through HTTP REST endpoints so other applications can request predictions. It supports request validation with Pydantic and provides interactive API documentation through Swagger UI. This makes the model easier to test and integrate with other software.

4. Why should the same trained model be reused for all inference strategies?

Reusing the same saved pipeline keeps preprocessing and prediction behaviour consistent across online, batch, streaming and API inference. It avoids unnecessary retraining and makes the outputs of the different inference methods comparable. The saved pipeline also ensures that the same fitted preprocessing transformations are applied during inference.

8. Results and Verification

The project is considered successfully demonstrated when the training script reports the held-out test accuracy, the online script produces one prediction, the batch output contains 20 predictions, the streaming script processes 10 records, and the FastAPI health and prediction endpoints respond as expected.

Actual test accuracy: Record the percentage printed by your own run of train_model.py.

API verification: Record the observed status codes from your own /health and /predict tests. Do not report a test as successful unless you have run it.

9. Conclusion

This assignment demonstrates the end-to-end workflow of training a machine learning classification pipeline, evaluating it on held-out data, saving it with Joblib, and reusing it for online, batch and streaming inference. It also demonstrates how FastAPI and Pydantic can expose the saved model through validated REST endpoints. Using one saved pipeline across all inference methods supports consistent preprocessing and prediction behaviour.

Note: The report describes the implementation specified for this assignment. Actual accuracy and API test results must be taken from the student's own execution.